DAY 09 / 15 · PHASE 3: SECURITY · 23 OCTOBER 2026

Smart-contract security: break it, then fix it

Five classic bugs, each with a working exploit and a fix. Study guide by Manish Khadka.

1. Learning objectives

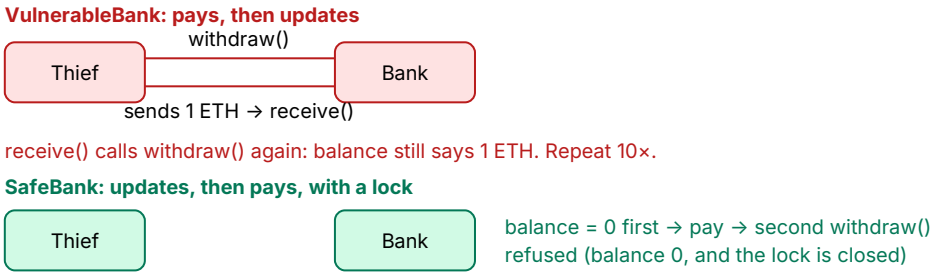
- Explain in plain words why a smart contract must survive **hostile** callers, not just friendly ones.
- Recognise and exploit five bugs: re-entrancy, missing access control, `tx.origin` authentication, front-running and denial of service.
- Apply the fix for each: checks-effects-interactions + lock, `onlyOwner` / `Ownable2Step`, `msg.sender`, commit–reveal, pull payments.
- Write **invariant (fuzz) tests** that find bugs you did not know to look for.

2. The concept, explained simply

A smart contract is a vending machine that holds real money and that **anyone in the world can poke, in any order, as often as they like**. It cannot be patched after deployment. So the question is never "does it work when I use it?" but "what if someone is trying to break it?"

#	Bug	Everyday picture	Fix
1	Re-entrancy	Cashier hands over cash *before* writing it in the book; you ask again while the book is still old.	Update the book first (CEI) + `nonReentrant` lock
2	Missing access control	The "make me owner" button has no lock.	Owner set once in the constructor; `onlyOwner`; two-step hand-over
3	`tx.origin`	Bank checks who started the errand, not who stands at the counter.	Check `msg.sender`
4	Front-running	You shout the quiz answer in a crowded room; someone faster repeats it.	Commit–reveal (sealed envelope)
5	Denial of service	A shop that must refund the last customer first; that customer refuses it.	Pull payments

Step by step: re-entrancy



The thief deposited **1 ETH** and walked away with **10 ETH**; the other 9 ETH belonged to Alice and Bob.

Step by step: front-running

1. Alice sends `solve("satoshi")` with a 2 gwei tip. It waits in the public **mempool**.
2. A bot reads it, copies the call and sends it with a 50 gwei tip.
3. Builders order by fee: the bot's copy runs first and wins 2 ETH; Alice's fails with `AlreadySolved`.
4. **Fix:** Alice first sends `keccak256(her address, answer, salt)` (a sealed envelope), then reveals one block later. The hash includes her address, so a copied reveal does not match the bot's own (non-existent) commitment.

3. Key code walkthrough

The fix for re-entrancy (`SafeBank.withdraw`)

```
function withdraw() external nonReentrant {          // lock: no second entry
    uint256 amount = balanceOf[msg.sender];          // CHECKS
    if (amount == 0) revert NothingToWithdraw();
    balanceOf[msg.sender] = 0;                       // EFFECTS: record first
    emit Withdrawn(msg.sender, amount);
    (bool ok, ) = msg.sender.call{value: amount}("");// INTERACTION: last
    if (!ok) revert TransferFailed();
}
```

The fix for front-running (`CommitRevealPuzzle`)

```
function makeCommitment(address player, string calldata answer, bytes32 salt)
    public pure returns (bytes32) {
    return keccak256(abi.encode(player, answer, salt)); // bound to YOUR address
}
function reveal(string calldata answer, bytes32 salt) external {
    Commitment memory c = commitments[msg.sender];
    if (c.hash == bytes32(0)) revert NoCommitment();
    if (block.number <= c.blockNumber) revert RevealTooEarly(); // at least 1 block later
    if (makeCommitment(msg.sender, answer, salt) != c.hash) revert CommitmentMismatch();
    ...
}
```

The fix for denial of service (`SafeKingOfTheHill.claimThrone`)

```
if (msg.value <= prize) revert BidTooLow(prize, msg.value);
if (king != address(0)) pendingReturns[king] += prize; // RECORD the refund, don't send it
king = msg.sender; prize = msg.value;
```

4. Results from my run

Test	Broken contract	Fixed contract
Re-entrancy (1 ETH deposit; 10 ETH in the bank)	10 re-entries, bank left with 0 ETH, thief profit **10 ETH**	1 entry, 1 refused, bank keeps 10 ETH, profit 0
Cost of safety	withdraw 28,479 gas	32,303 gas (+3,824, about 13%)
Open `initialize()`	Thief becomes owner, wallet emptied	`OwnableUnauthorizedAccount`; wallet keeps 5 ETH
`tx.origin` phishing	Alice clicks "claim"; wallet emptied	Refused (`NotOwner`); 5 ETH stays
Front-running	Bot at position 0 wins 2 ETH; Alice at 1 fails	Bot's copy fails (`NoCommitment`); Alice wins
DoS (refund always fails)	King stuck for ever (`RefundFailed`)	Carol becomes king; the stubborn one only blocks itself

Tests: 31 / 31 passing, including fuzz invariants (200 random programs, 1,257 steps, "ETH held = sum of balances" held after every step).

5. QA & security findings

ID	Sev.	What	Fix
M1	Medium	`SafeWallet` inherited `renounceOwnership()`: one call would lock all ETH for ever.	Override to revert (`RenounceDisabled`)
S1	Low	A `transfer(address,uint256)` without a return value looks like an ERC-20 token (Slither `erc20-interface`).	Renamed `transferBalance` / `pay`
T1	Low	My cross-function re-entrancy test passed for the wrong reason (balance was already 0).	Assert the exact `ReentrancyGuardReentrantCall` error
E1	Low	Evening audit: `transferBalance(bankAddress, x)` parked money where nobody could ever withdraw it.	Revert `InvalidRecipient`

Also: Slither flagged only 2 of my 5 planted bugs (re-entrancy, `tx.origin`). The open `initialize()`, the front-running and the stuck refund were invisible to it: **tools help, exploit tests**

prove. `npm audit`: 0 vulnerabilities. Known issues: weak answers can be guessed offline (K1), a producer can censor a reveal (K2).

6. Common mistakes & 5 interview questions

- Sending ETH before updating your own records.
- Leaving an admin or `initialize()` function open.
- Using `tx.origin` to decide who is allowed.
- Trusting `address(this).balance` as bookkeeping (ETH can be forced in with `selfdestruct`).
- Looping over users or pushing refunds inside someone else's transaction.
- Believing a test that "passes" proves the lock works (my T1).

Q1. What is re-entrancy and how do you stop it? A contract is called again before its first call has finished, while its records are still old. Follow checks-effects-interactions and add a `nonReentrant` lock on every function that moves value.

Q2. Why is `tx.origin` dangerous for authorisation? It is the person who signed the whole transaction, even if a malicious contract is the one calling. A phishing contract the owner calls passes the check. `msg.sender` is the immediate caller and fails for the phisher.

Q3. How does commit–reveal beat front-running? The commitment hides the answer and is bound to the sender's address; the reveal must come in a later block. A bot copying the reveal has no earlier matching commitment of its own.

Q4. Push vs pull payments? Push sends money inside your action, so one refusing receiver can block everyone. Pull records the debt and lets each person collect their own.

Q5. Why use invariant/fuzz tests? An example test only checks cases you thought of. An invariant ("ETH held = sum of balances") checked after random sequences finds bugs you did not know existed; my fuzzer broke a bank with one deleted line in 2 steps.

7. Exercises summary

Level	Task
Easy	Build a commit–reveal commitment by hand in JavaScript and compare with `makeCommitment`; list contracts that look like ERC-20. Why `abi.encode` and not packed encoding?
Medium	Write a pull-payment auction; prove a bidder that cannot receive ETH does not block it (`PullAuction`).
Challenge	Fuzz `LeakyBank` (one deleted line) with the invariant "held = sum of balances"; get the shortest failing program.

Cheat sheet

- Order: **Checks → Effects → Interactions** + `nonReentrant`.
- Every function is public to the world unless locked: `onlyOwner`, `Ownable2Step`, disable `renounceOwnership` when funds are held.
- Authorise with `msg.sender`, never `tx.origin`.
- Mempool is public: hide secrets with **commit–reveal**.
- Never let one receiver block others: **pull, don't push**.
- Keep your own records; do not use `address(this).balance` for accounting.
- Ask "where can money get stuck?" and write an invariant.

8. References

1. Solidity docs, Security considerations: https://docs.soliditylang.org/en/v0.8.24/security-considerations.html
2. SWC Registry: https://swcregistry.io/
3. ethereum.org, Smart contract security: https://ethereum.org/en/developers/docs/smart-contracts/security/
4. ethereum.org, Maximal extractable value (MEV): https://ethereum.org/en/developers/docs/mev/
5. OpenZeppelin Contracts 5.x, Access control: https://docs.openzeppelin.com/contracts/5.x/access-control
6. OpenZeppelin Contracts 5.x, API utilities (ReentrancyGuard): https://docs.openzeppelin.com/contracts/5.x/api/utils
7. Slither static analyzer (Trail of Bits / Crytic): https://github.com/crytic/slither
8. Consensys, Smart Contract Best Practices: https://consensysdiligence.github.io/smart-contract-best-practices/
9. fast-check, property-based testing: https://fast-check.dev/
10. Code for this day: https://github.com/Khadka1996/Learning-Blockchain/tree/main/day-09